Shared Household Journal — Technical Requirements Document

Claude Code in VS Code Edition | Repository-first technical guide | Draft v2

Document status and precedence

Draft v2. This document describes technical intent and safeguards for Claude Code running in VS Code. It is not a mandate to replace the repository's existing stack. First inspect the actual codebase; document discrepancies; then adapt this TRD to the verified architecture. Existing stable implementation choices should be preserved unless a change is approved.

1. Suggested architecture (verify first)

Preferred baseline when the repository is greenfield: Next.js with TypeScript; Supabase Auth, Postgres, and Row Level Security; Tailwind CSS; an established component library such as shadcn/ui if already installed; Zod or equivalent schema validation; server-side data access for sensitive operations; and automated tests for domain logic.

This is a recommendation, not an instruction to install all packages. Claude Code must inspect package.json, lockfiles, source structure, environment templates, and existing libraries before proposing dependencies.

2. Repository discovery checklist

Before coding, inspect:

• git status and current branch; do not discard user changes.

• package.json scripts, lockfile, framework and dependency versions.

• app/src/pages/routes, shared components, styles, and design tokens.

• auth setup, Supabase clients, database schema/migrations, generated types.

• existing RLS policies and server/API boundaries.

• test framework, lint/typecheck/build scripts, and current failures.

• .env.example or equivalent variable names only; never read out or expose secret values.

• README, project notes, and deployment configuration.

Deliver a concise audit: current architecture, functioning features, missing requirements, risks, commands available, and phase-specific plan. If repository state is ambiguous, ask before destructive action.

3. Data and domain requirements

Core entities (adapt names to existing schema): profiles; households; household_members; invitations; expenses; expense_splits; settlements; recurring_bills; activity_events; shopping_items; chores; budgets; notification_preferences.

Use stable IDs, created/updated timestamps, household foreign keys, and explicit ownership/membership relationships. Money should use integer minor units or an exact decimal representation—not binary floating-point as the source of truth. Define currency precision and rounding rules. Preserve an auditable relationship between an expense, its splits, and resulting balances. Balance summaries should be derived or safely cached with a reconciliation path.

4. Security and authorization

• Treat household membership as the authorization boundary.

• Enforce authorization in database RLS and/or trusted server-side logic; UI hiding is not security.

• Use least privilege for service-role credentials; never expose privileged keys to browser code.

• Validate all inputs at the server/API boundary, including household ID and member access.

• Check access on reads, inserts, updates, deletes, and storage objects if used.

• Never put secrets in source code, prompts, logs, screenshots, commits, or generated docs.

• Use only environment variable names in documentation; never print actual values.

• Invite tokens should be unguessable, time-limited where appropriate, and single-use or otherwise safely managed.

• Add tests or documented manual verification for cross-household access denial.

5. Money logic and invariants

• Amounts must be positive unless a clearly defined correction/refund workflow permits otherwise.

• Equal splits use deterministic remainder distribution at currency precision.

• Percentage splits must total 100% within the agreed tolerance.

• Custom splits must sum exactly to the expense total in minor units.

• Expense payer and split participants must be valid household members under the selected policy.

• A settlement reduces the relevant net obligation only once.

• Editing/deleting an expense must recalculate affected derived results consistently.

• Never trust client-calculated balances as authoritative.

• Create unit tests for rounding, zero/one participant, payer included/excluded, partial settlement, multiple settlements, edits, deletions, and concurrent updates where applicable.

6. Database and migration policy

Migrations must be additive and reviewable. Claude Code must inspect the current migration history before proposing changes. Do not reset, drop, truncate, or rewrite production data. For destructive or data-transforming changes: explain impact, provide a reversible/back-up plan where possible, and request explicit approval. Keep generated database types synchronized using the repository's established workflow. Validate RLS policy changes against existing tables and access paths.

7. UI implementation constraints

Follow the existing routing, form, component, and styling conventions. Reuse installed components and design tokens. Avoid duplicate component systems. Build responsive layouts and accessible interaction states. Use clear confirmation for destructive actions and show validation near the affected field. Keep financial details understandable, with links to source expenses/settlements where feasible. Do not add motion that harms reduced-motion users.

8. Testing and quality gates

Use only commands discovered in the repository or documented by the project. Run relevant tests after each meaningful change and broader checks at phase completion. Typical gates, if available: unit tests, integration tests, lint, typecheck, build, RLS/security checks, and manual responsive verification.

Do not invent test results. If a command cannot run because a dependency, credential, or environment is unavailable, report it as not run and explain why. Do not alter tests simply to make them pass without validating the intended behavior.

9. Claude Code workflow in VS Code

1. Open the project root in VS Code and launch Claude Code from that workspace.

2. Ensure Git is initialized and inspect the working tree; commit or back up important work before large changes.

3. Start with a repository audit prompt. Do not ask Claude Code to implement the entire application at once.

4. Work one phase and one bounded task at a time.

5. Review diffs in VS Code Source Control before accepting changes.

6. Run the project's real checks in the integrated terminal.

7. Ask Claude Code to summarize changed files, decisions, verification, unresolved issues, and next task.

8. Keep CLAUDE.md concise and project-specific; it should state commands, architecture facts, coding conventions, safety rules, and phase boundaries. Do not duplicate the entire PRD/TRD in it.

10. Required reporting format

At the end of every task, Claude Code should report:

• Audit findings and assumptions confirmed.

• Files created/modified/deleted.

• User-visible behavior implemented.

• Data/schema/API changes and migration notes.

• Tests/checks actually executed and their observed results.

• Checks not run and why.

• Risks, limitations, or follow-up work.

• A concise recommended next task.

No claim of completion without evidence. Preserve pre-existing user changes and disclose any conflicts.

11. Non-functional requirements

Security: household isolation and safe secret handling.

Reliability: consistent money calculations and recoverable failures.

Usability: mobile-first core workflows, readable hierarchy, meaningful empty/error states.

Maintainability: typed domain logic, small modules, explicit validation, synchronized docs.

Performance: paginate or constrain large activity/expense queries; avoid unnecessary client fetches.

Accessibility: semantic structure, keyboard support, visible focus, labels, contrast, reduced motion.

Observability: useful non-sensitive error logging; never log credentials or unnecessary personal/financial data.

12. Unconfirmed technical decisions

Do not silently decide: exact currency/multi-currency support; soft-delete vs hard-delete; invitation expiry; settlement reversal rules; timezone and recurrence semantics; notification provider; OCR/AI provider and data handling; export/retention policy; production hosting; and any new dependency that materially changes architecture.